

# File handling

Revise 360 lesson plan — 60 minutes

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Unit            | Beyond OCR J277: a Python pathway supporting 2.1, 2.2 and 2.3                                                     |
| Lesson          | File handling (lesson 10 of the unit)                                                                             |
| Lesson ID       | pr-l10                                                                                                            |
| Experience ID   | pr-l10                                                                                                            |
| Plan ID         | PLAN-PR-L10                                                                                                       |
| Content version | 15b78a341706                                                                                                      |
| Website         | <a href="https://revise360.co.uk/experience.html?id=pr-l10">https://revise360.co.uk/experience.html?id=pr-l10</a> |

## Big question, outcomes and success criteria

**Big question** — Why does the mode a file is opened in matter more than anything else on that line?

### Prerequisites

The lesson before this one is pr-l09. Pupils should already have finished it: the starter below retrieves from it.

Earlier in the unit: pr-l01, pr-l02, pr-l03, pr-l04, pr-l05, pr-l06, pr-l07, pr-l08.

### Assessable outcomes

1. **pr-l10-o1** — know how to open a text file, read what is in it and close it again
2. **pr-l10-o2** — understand what write mode does to a file, and how append mode differs from it
3. **pr-l10-o3** — know how to split one line of a file into its separate fields and use them

### Success criteria the pupil can check against

Taken from the briefs the independent activities themselves show, so these are the words the pupil sees. Their work has met the outcome when it does these things.

- Change only the last line. **pr-l10-s1-a3**
- Display what was read and nothing of your own. **pr-l10-s1-a3**
- Replace \_\_\_\_ with one method of the file. **pr-l10-s1-a4**
- Display the whole file and nothing of your own. **pr-l10-s1-a4**
- Nothing is typed in. **pr-l10-s1-a5**
- The program only reads `"notes.txt"`. **pr-l10-s1-a5**
- Display nothing of your own, not even a heading. **pr-l10-s1-a5**
- Display the contents of the file and nothing else. **pr-l10-s1-a6**

## Vocabulary, resources and preparation

**Key vocabulary** — Append mode, Record, Field, split()

### Resources

|                   |                                               |
|-------------------|-----------------------------------------------|
| SLIDES            | presentations/PR_L10_FileHandling_Lesson.pptx |
| WORKSHEET         | worksheets/PR_L10_FileHandling_Worksheet.docx |
| ONLINE EXPERIENCE | experience.html?id=pr-l10                     |

## Before the lesson

1. Sign in on the **teacher dashboard** (`teacher.html`) and open **Class logins**. Enter the class or group and the usernames, one per line, then print the login cards. A pupil cannot sign in without one, so do this before the lesson rather than in it.
2. Print the worksheet, one per pupil.
3. Open the slides and the experience once yourself, on the machine you will be teaching from, so the Python runtime and the 360 image are already cached.
4. Check headsets or devices are charged and on the school network.

## How pupils get in

1. Go to `topics.html` on `revise360.co.uk`.
2. Sign in with the **username and PIN from the login card**. The class is already set against the card.
3. Choose the topic, then this lesson's card.

**Worth knowing before you promise anything.** In this build `APP_CONFIG.backendUrl` is empty, so logins, login cards and progress are stored on the device the pupil used. The teacher dashboard says so too. A pupil who moves to a different machine starts again, so keep a class on the same devices within a unit, and treat the paper worksheet as the record that travels.

## Teaching sequence

Sixty minutes including transitions. The online block is the one to move if your lesson is shorter: see the adaptation at the end.

| MIN   | PHASE                                     | TEACHER                                                                                      | PUPILS                                                       | EVIDENCE TO CHECK                                                                |
|-------|-------------------------------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------|
| 0-5   | <b>Retrieve</b>                           | Starter on the board or the worksheet, from the last lesson and this lesson's prerequisites. | Answer the starter on the worksheet.                         | The starter box on <code>PR_L10_FileHandling_Worksheet.docx</code> is filled in. |
| 5-8   | <b>Review the starter</b>                 | Take answers; correct the one most got wrong before moving on.                               | Correct their own starter.                                   | A correction written, not just a tick.                                           |
| 8-16  | <b>Explain and model</b>                  | The new idea, then the worked example, typed live rather than shown finished.                | Watch the example being built, and say what each line does.  | They can name the line that does the job, not just copy it.                      |
| 16-19 | <b>Set up and settle</b>                  | Sign-in, launch, and what to record on paper before anyone types.                            | Sign in, open the lesson, read the task before typing.       | Everyone is at station 1 activity 1, not typing yet.                             |
| 19-48 | <b>Online: work the stations in order</b> | Pupils work; you circulate and read the editor over shoulders.                               | Work the stations in order, running and checking as they go. | The station badges fill in; the editor shows their own program, not the starter. |
| 48-53 | <b>Check understanding</b>                | The hinge question, hands down, then the reteach if the class splits.                        | Answer the hinge question, hands down.                       | The split across the options, before any reteach.                                |
| 53-58 | <b>Apply or extend</b>                    | The challenge activity, or the exam practice on the worksheet.                               | The Challenge activity, or the exam practice.                | worksheet, Exam practice a) to c)                                                |
| 58-60 | <b>Exit and set up the next lesson</b>    | The exit question and what to finish before next time.                                       | Write the exit answer.                                       | One sentence that answers the big question.                                      |

## The explanation and the worked example

---

This is what the walls of the experience say, station by station. Teach the first two or three from the front before anyone puts a headset on; the rest the pupils read for themselves.

### 1. Reading a whole file

- `open(name, "r")` opens a file for reading and hands back a file object.
- `.read()` gives the whole file as one string, and `.close()` lets go of it when you are done.
- Common mistake: opening a file that is not there, which stops the program with an error.

**Challenge on this wall:** Why should a program close a file as soon as it has finished with it?

### 2. Reading line by line

- `.readline()` reads one line, while `.readlines()` gives a list of every line.
- Each line still carries its newline, so `.strip()` tidies that off before you use it.
- Common mistake: measuring a line before stripping it, which counts the newline as a character.

**Challenge on this wall:** Why is a stripped line one character shorter than the line that was read?

### 3. Writing to a file

- `open(name, "w")` opens a file for writing and empties whatever was in it.
- `.write()` adds no newline of its own, so put a newline on the end of each line yourself.
- Common mistake: using write mode on a file you meant to add to, which wipes it completely.

**Challenge on this wall:** What happens to the contents of a file the moment it is opened in w mode?

### 4. Adding to a file

- `open(name, "a")` is append mode: new text goes on the end and nothing already there is lost.
- Use it whenever the lines the file already holds have to survive the run.
- Common mistake: leaving the newline out, so the new text runs straight into the last line.

**Challenge on this wall:** Which mode keeps what a file already holds, and why would you not use the other one?

### 5. Records in a file

- A record is one line of the file, and its fields are the parts separated by a comma.
- `.split(",")` breaks a line into a list of its fields.
- Common mistake: splitting before stripping, which leaves a newline stuck on the last field.

**Challenge on this wall:** How does a program find the one record whose first field matches what was typed in?

### 6. Putting it together

- Read the whole file first, close it, then work on the lines you are holding.
- A program can read one file and write a second one in the same run.
- Common mistake: looping over a file you have already closed, or reading and writing one file at once.

**Challenge on this wall:** Which of two files does a program open in which mode, when it reads one and writes the other?

## The example to model

A file lets a program keep something after it has stopped. `open()` needs the file name and a mode. The mode "w" means write and "r" means read. `.read()` brings the whole file back as one string, and `.close()` finishes with the file.

```
f = open("notes.txt", "w")
f.write("red\n")
f.write("blue\n")
f.close()
f = open("notes.txt", "r")
text = f.read()
f.close()
print(text)
```

It displays **red / blue**

**Decisions to model out loud, not just the finished answer:** say why you chose that name, why that line comes before the next one, and what you would try first if it did not work. The example deliberately uses different data from the task, so a pupil cannot copy it and be finished.

## Retrieval and the check for understanding

**Starter** — A program opens scores.txt in write mode, and the file already holds 40 lines. Write down what the file holds the moment that line has run, and which mode should have been used to add to it instead.

Answer space: 3 lines on the worksheet.

The answer is in the teacher answers for this unit, not on this plan: a plan left on a desk should not give it away.

### Hinge question

Use this one, hands down, before anyone moves on to independent work. It is a real activity from the experience (pr-110-s4-c2), so the wording a pupil sees matches the wording you say.

#### What will this program display?

```
f = open("notes.txt", "w")
f.write("ridge\n")
f.close()
f = open("notes.txt", "a")
f.write("hollow\n")
f.close()
f = open("notes.txt", "w")
f.write("moss\n")
f.close()
f = open("notes.txt", "r")
print(f.read())
f.close()
```

**Correct:** moss

#### What a pupil might choose instead

- ridge hollow moss
- ridge hollow

Count the hands on each. If more than a few of the class are on one wrong option, that option is the misconception to teach against before anyone goes on: go back to the station this question sits on and work through its example again, aloud, before the independent work.

**The course's own response to a wrong answer here:** The last open() used "w", which emptied the file, so ridge and hollow were gone before moss was written.

**One response, not four.** one response covers every wrong option; it does not say what to revisit for each one, so decide your own reteach for each option before the lesson - the list above is the start of it.

#### Misconceptions this lesson addresses

- opening a file that is not there, which stops the program with an error. [wall panel](#)
- f is the link to the file, not the words inside it. Something has to fetch the words. [pr-110-s1-a6, a Fix it activity](#)
- measuring a line before stripping it, which counts the newline as a character. [wall panel](#)
- Read the error. Every line that starts a block ends with the same punctuation mark. [pr-110-s2-a6, a Fix it activity](#)
- using write mode on a file you meant to add to, which wipes it completely. [wall panel](#)
- Read the error. A file opened for reading will not let anything be written to it. [pr-110-s3-a6, a Fix it activity](#)
- leaving the newline out, so the new text runs straight into the last line. [wall panel](#)
- What is written is only really in the file once the file has been closed. [pr-110-s4-a6, a Fix it activity](#)

## Guided to independent practice

| STAGE     | ACTIVITIES | WHAT IT IS                                                                                                                |
|-----------|------------|---------------------------------------------------------------------------------------------------------------------------|
| Recognise | 12         | The answer is on the screen and the pupil picks it: multiple choice, choosing several, matching a pair.                   |
| Construct | 12         | Nothing is offered. The pupil classifies, orders, fills in a table or works a number out, and types or places the answer. |
| Produce   | 13         | The pupil writes it from the brief and nothing else.                                                                      |
| Challenge | 6          | The hardest activity on each station. It is not optional and it is not extra: it is the last step of the station.         |

### Scaffolds already in the product, and how they fade

- **Hint** is a ladder, not a door: the first rung names the idea, the next shows the shape of the code, the next how it starts, the last animates the technique on different data. A pupil asks for each rung. None of them is the answer.
- **Run** is free and unmarked; **Check my answer** marks. A pupil may run as often as they like, which is what makes trial and error safe.
- **I need help** says to ask you, and says plainly that the question still has to be completed. It does not notify anyone and it does not unlock anything.
- The worked example appears on every early activity and on none of the late ones. That fade is deliberate; it is not a missing example.

### Accessibility

- **Read aloud** on every task window reads the task and its brief, with the prescribed values read as values rather than as punctuation.
- Text is 16px or larger throughout, and a question window scrolls rather than shrinking its type.
- Prescribed data is boxed and in a monospace face as well as coloured, so it still reads as code in black and white, in a high-contrast mode, and to a pupil who does not separate green from amber.
- **Lower literacy and EAL:** pre-teach the vocabulary above; let the pupil say the key fact before writing it; accept the key fact in two or three words.
- **SEND:** the sequence is compulsory but the pace is not. A pupil may finish a station next lesson; the progress saves after every answer and reopens at their earliest unfinished question.
- A headset is never required. Every experience works on a laptop or a tablet with a mouse or a finger.

## Delivering the online experience

### Before anyone enters

- Worksheet out, pen in hand. The rule is write first, then answer at the badge: the first answer is the one that counts.
- Say where the stations are. They are numbered around the room: 1 and 2 to the right, 3 and 4 behind, 5 and 6 to the left, and the final challenge on the floor.
- Say that a badge marked **Not yet** is not a fault. It opens when the station they are on is finished.

### The stations, and what to record at each

| # | STATION              | ACTIVITIES | RECORD ON PAPER                                                                                                |
|---|----------------------|------------|----------------------------------------------------------------------------------------------------------------|
| 1 | Reading a whole file | 7          | Leaving a file open can keep other programs out of it, and can lose anything not yet...                        |
| 2 | Reading line by line | 7          | A newline is one single invisible character, which is why a stripped line is one character...                  |
| 3 | Writing to a file    | 7          | Write mode makes the file when it is not there and empties it when it is, so the one thing it never does is... |
| 4 | Adding to a file     | 7          | Three modes cover everything you need: r reads, w starts the file again from empty and a...                    |

| #     | STATION                      | ACTIVITIES | RECORD ON PAPER                                                                                                |
|-------|------------------------------|------------|----------------------------------------------------------------------------------------------------------------|
| 5     | Records in a file            | 7          | A file of comma-separated records has a name of its own, and a spreadsheet opens one straight away: it is a... |
| 6     | Putting it together          | 7          | A stray blank line at the end of a file is very common, so a careful program checks each line before it...     |
| final | Final challenge: which mode? | 1          | the answer to the station's challenge                                                                          |

## Where to pause

Stop the class after station 2 and take answers to its challenge out loud. That is the point where a pupil who has misread the technique can still be put right cheaply.

## Afterwards

Headsets off and down. Ask three pupils for the key fact from a station of your choosing, in their own words and not read off the sheet. Then the exit question.

**Python: the order is not negotiable.** Each activity teaches what the next one needs, so a pupil works through them in order and completes every one. A shorter lesson, homework or a slow worker does not authorise skipping: the pupil stops where they are and resumes at their earliest unfinished question next time, because progress saves after every answer.

A paper activity, a help request or a tick on this plan does not mark an online question complete, and there is no teacher bypass. If a pupil is stuck after the hint, that is the moment to teach them, not to move them on.

## Assessment, evidence and unfinished work

| ID        | QUESTION                                                                                                                                                                                              | MARKS | WHERE                                                  |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|--------------------------------------------------------|
| pr-l10-x1 | Every line read from a file is measured with <code>len()</code> , and each answer comes out one character bigger than expected. Name the character being counted, and write the line that removes it. | 2     | worksheet, Exam practice a); deck, Exam practice slide |
| pr-l10-x2 | Write a program that reads a word and adds it onto the end of <code>"log.txt"</code> on a line of its own, without losing the lines already in the file, then prints every line of the file.          | 4     | worksheet, Exam practice b); deck, Exam practice slide |
| pr-l10-x3 | Each line of <code>"data.txt"</code> holds a word and a whole number, separated by a comma. Write a program that prints the word from every line whose number is 16 or more.                          | 5     | worksheet, Exam practice c); deck, Exam practice slide |

## Expected answers

Model answers and marking guidance are teacher-only and are not printed on this plan. They are in the unit's teacher guide pack, and the per-activity marking for this lesson is in `answers/records/pr-l10.json`, which is not published with the website.

Every programming activity in this lesson is marked by running the pupil's program against its test cases, so the mark is the program's behaviour and not its appearance. The model solutions are in `answers/codebank/pr-l10.json`.

## Common errors

The wrong options the activities offer are the errors this lesson expects. The commonest to watch for:

- **What will this program display?** — ridgehollow; ridge hollow
- **What will this program display?** — ridge ridge; hollow ridge
- **What will this program display?** — ridge hollow; ridge hollow
- **What will this program display?** — ridge hollow moss; ridge hollow
- **What will this program display?** — 2 1; 3 1

## Unfinished work

Record where each pupil stopped, not a tick. They resume at their earliest unfinished question, and a station left half-done is the thing to open next lesson. Do not mark the lesson complete for a pupil who did not finish it: the score screen already shows what was and was not done, and that is the honest record.

## **Exit, next lesson and homework**

---

### **Exit check**

One of these, written, not a show of hands:

- Opening, reading and closing a file
- Choosing between write mode and append mode
- Splitting a record into its fields

The confidence columns on the worksheet are a supplement to this, not a substitute: a pupil who rates themselves confident and cannot answer the exit question has told you nothing.

### **Retrieval link to next lesson**

Next is pr-111, Searching and sorting. Use this lesson's big question as next lesson's starter, so the first thing they do is retrieve it.

### **Homework**

Finish any station left open, on the website, from the question they stopped at. If the class has no devices at home, set the exam practice on the worksheet instead and mark it next lesson.

## **Shorter lessons, longer lessons and no internet**

---

### **If this lesson takes more than an hour**

It will. There are 43 activities across 6 stations, and a beginner does not finish 43 of them in twenty-nine minutes. Plan it as two or three sessions of an hour: stations 1 and 2 in the first, 3 and 4 in the second, 5, 6 and the final challenge in the third. Each session keeps the same starter-explain-model opening; the online block resumes where the class got to. Honest total: two to three hours.

### **A 40 to 45 minute lesson**

Keep the starter, the explanation and the model at full length: cutting those is what makes the online work fail. Take the time out of the online block and the apply phase. Pupils stop where they are; their progress saves after every answer and reopens at their earliest unfinished question, so the next lesson starts there.

A shorter lesson still does not authorise skipping a question. The save-and-resume point is wherever the pupil got to; there is no way to mark a question complete without completing it, and no teacher bypass.

### **If the internet or the devices are unavailable**

Teach from the slides and the worksheet. The slides carry the same teaching as the station walls, and the worksheet carries the key facts, the challenges and the exam practice. Set the online stations when the devices are back.

There is no offline version of the experience, and this plan does not contain one. Do not hand out a printed walkthrough of the stations as a substitute: the experiences are used on the website.

---

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.